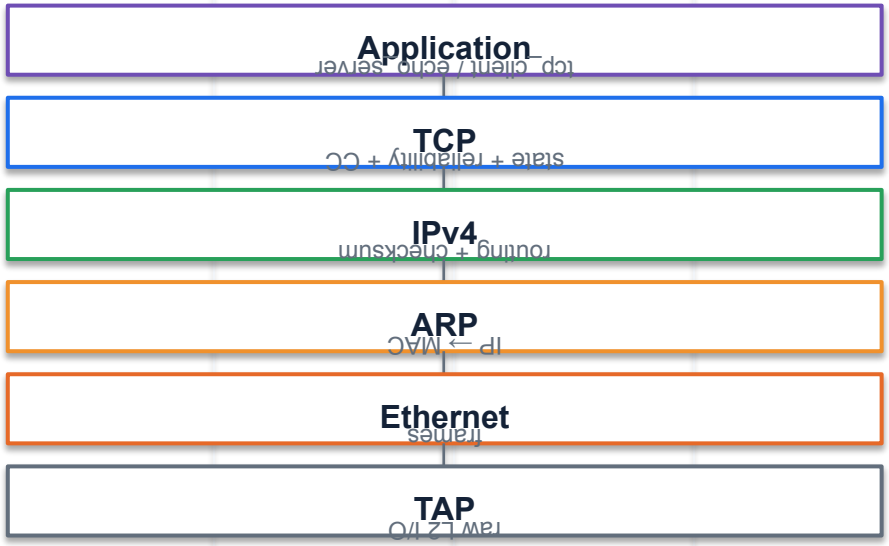
NETWORKING SYSTEM PROJECT

Custom TCP/IP Stack with Congestion Control

A user-space networking stack in C built from Ethernet/TAP upward to TCP, with reliability, retransmission, RTT estimation and Reno-style congestion control.



Core idea

Build the network stack ourselves instead of relying on the kernel's TCP implementation.

Proof

Real Ethernet frames through a TAP device; inspectable with packet capture.

Experiment

Measure cwnd, RTT, retransmissions and throughput under loss.

Professor-facing narrative: architecture → packet flow → TCP state → congestion control → live observability.

1. Project objective

What the system is designed to demonstrate

Build from the wire upward

Access raw Ethernet frames through TAP and implement the lower networking layers in C.

Implement TCP behavior

Handshake, ordered byte stream, ACKs, send/receive windows, retransmission and connection teardown.

Make congestion visible

Track cwnd, ssthresh, RTT, RTO, duplicate ACKs, loss events and throughput so TCP behavior can be measured.

Scope

- Ethernet II frame parsing and construction
- ARP cache and request/reply handling
- IPv4 parsing, routing decision and checksum
- ICMP echo and UDP port callbacks
- TCP state machine and connection table
- Reno-style congestion-control model

Success criterion

Working demonstration

A custom TCP client should establish a real connection, transfer application data, survive controlled loss, and expose measurable congestion-control behavior.

2. Why TAP instead of ordinary sockets?

The design choice that makes the project a real stack

Normal socket API

Application → socket() → Linux TCP/IP → NIC

The operating system owns TCP, IP, ARP and Ethernet processing.

TAP-based custom stack

Application → custom TCP → IPv4 → ARP/Ethernet → TAP → Linux network

The project owns the protocol logic in user space.

What TAP gives the program

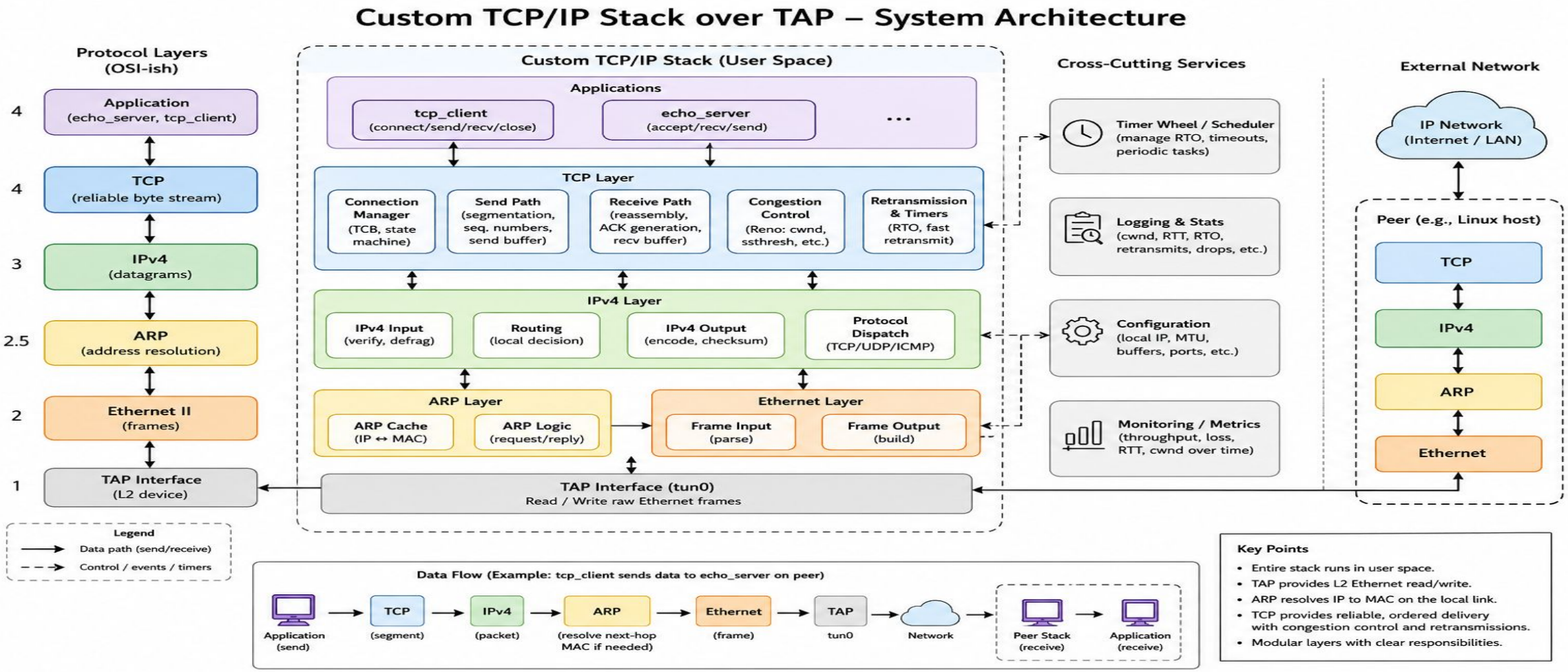
- Layer-2 Ethernet frames rather than pre-parsed IP packets
- Read/write interface for the custom stack
- Ability to observe and construct actual on-wire headers

What the project must now implement

- Frame parsing and EtherType dispatch
- ARP resolution and cache management
- IPv4 validation/routing/checksum
- TCP reliability and congestion control

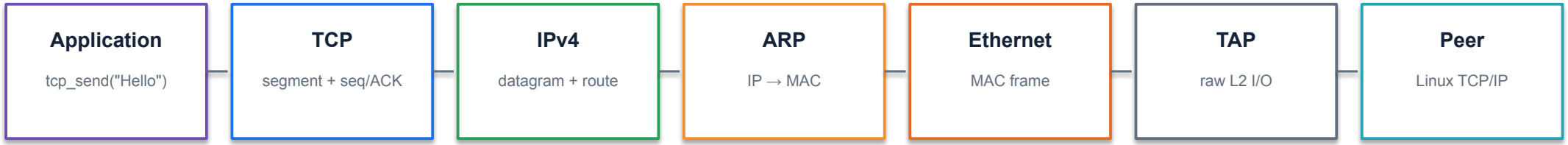
3. System architecture

Layered protocol stack + cross-cutting services



4. End-to-end packet path

One TCP send travels through every layer



Receive path is the reverse: TAP → Ethernet → IPv4 → TCP → application.

Send-side responsibility

Each layer adds its own header, validates constraints, computes required checksums and passes the resulting bytes downward.

Receive-side responsibility

Each layer parses only what it owns, validates lengths/checksums, then dispatches the payload upward.

Why layering matters

A TCP bug can be isolated from ARP/IP bugs; packet captures can be compared with the exact layer that produced them.

5. Ethernet + ARP + IPv4

The lower layers make TCP possible

Ethernet II

Header: destination MAC, source MAC, EtherType.

The receive dispatcher selects ARP or IPv4 based on EtherType.

```
ethertype = ntohs(hdr->ethertype);
if (ethertype == 0x0806) → ARP
if (ethertype == 0x0800) → IPv4
```

ARP

Maintains an IP → MAC cache. If a next-hop mapping is missing, the stack sends an ARP request and learns from replies.

```
ARP REQUEST: Who has 10.0.0.2?
ARP REPLY: 10.0.0.2 is aa:bb:cc:...
```

IPv4

Validates version/IHL, checksum, destination and fragmentation assumptions. Routes locally or via gateway, then resolves the next-hop MAC.

```
next_hop = same_subnet(dst)
    ? dst
    : gateway
```

Current implementation intentionally omits IPv4 fragmentation/reassembly and TCP options in the first scope. These should be documented as limitations rather than hidden.

6. TCP module architecture

Separate connection state, data movement and congestion control

TCP Control Block (TCB)

One TCB represents one 4-tuple connection.

Connection state

state, local/remote 4-tuple

Send state

snd_una, snd_nxt, snd_wnd, ISS

Receive state

rcv_nxt, rcv_wnd, IRS

Reliability

in-flight segments, ACK tracking

RTT/RTO

SRTT, RTTVAR, RTO, backoff

Congestion control

cwnd, ssthresh, phase, dup ACKs

Key principle: TCP is an event-driven state machine, not a blocking send/receive loop.

7. TCP three-way handshake

How a connection becomes ESTABLISHED



Active open

Client chooses an initial sequence number (ISS), sends SYN and waits for SYN+ACK.

Passive open

Listener creates a child TCB for the peer, records IRS/RCV.NXT and sends SYN+ACK.

Established

The final ACK confirms both directions' sequence spaces and enables application data transfer.

8. Data transfer + flow control

Congestion window and receive window jointly constrain sending

$$\texttt{effective_send_window} = \min(\texttt{cwnd}, \texttt{snd_wnd}) - \texttt{bytes_in_flight}$$

cwnd

Sender's congestion-control limit. It reflects how much data the network is believed to tolerate without causing excessive congestion.

snd_wnd

Receiver's advertised flow-control window. It protects the receiver's buffer from being overrun.

bytes in flight

snd_nxt − snd_una. These bytes have been transmitted but are not cumulatively acknowledged yet.

Example

```
cwnd = 8 KB
snd_wnd = 12 KB
flight = 5 KB
usable = min(8, 12) − 5 = 3 KB
```

Implementation note

- Incoming TCP window must update snd_wnd.
- Sending must stop when usable window reaches zero.
- Actual retransmission requires retaining segment bytes.

9. Retransmission + RTT estimation

Reliability is driven by ACKs and timers

Transmit

segment stored in flight

ACK

ACK advances snd_una

RTT sample

only if not retransmitted

RTO

SRTT + 4×RTTVAR

Timeout path

RTO expires

Mark the oldest unacknowledged segment as retransmitted, apply exponential backoff and reduce cwnd.

Retransmit

Resend the same sequence number and payload. Do not advance snd_nxt as if it were new data.

Karn's algorithm

Do not use RTT samples from retransmitted segments because the ACK cannot identify which transmission it measured.

RFC 6298-style estimator: initialize SRTT/RTTVAR from the first valid sample; update thereafter and back off RTO after timeouts.

10. Reno-style congestion control

The core experiment: how TCP reacts to congestion



Normal ACK

- Slow start: cwnd grows rapidly.
- Congestion avoidance: approximately +1 MSS per RTT.

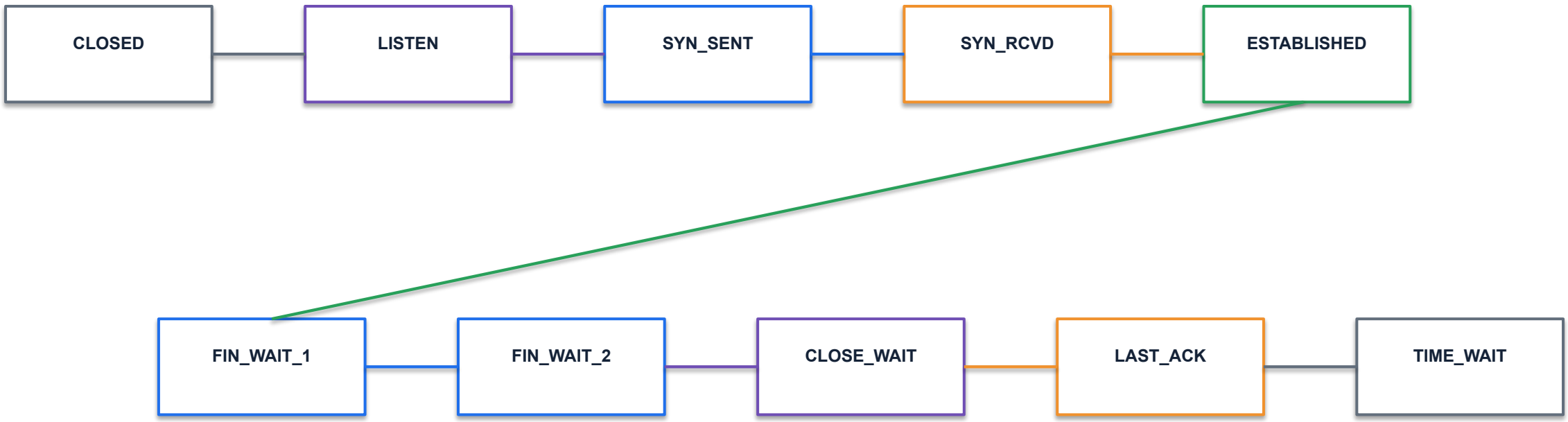
Loss response

- Timeout: ssthresh ≈ cwnd/2, cwnd → 1 MSS.
- 3 duplicate ACKs: fast retransmit + fast recovery.

Important distinction: timeout response and triple-duplicate-ACK response should use different congestion-control transitions.

11. TCP state machine

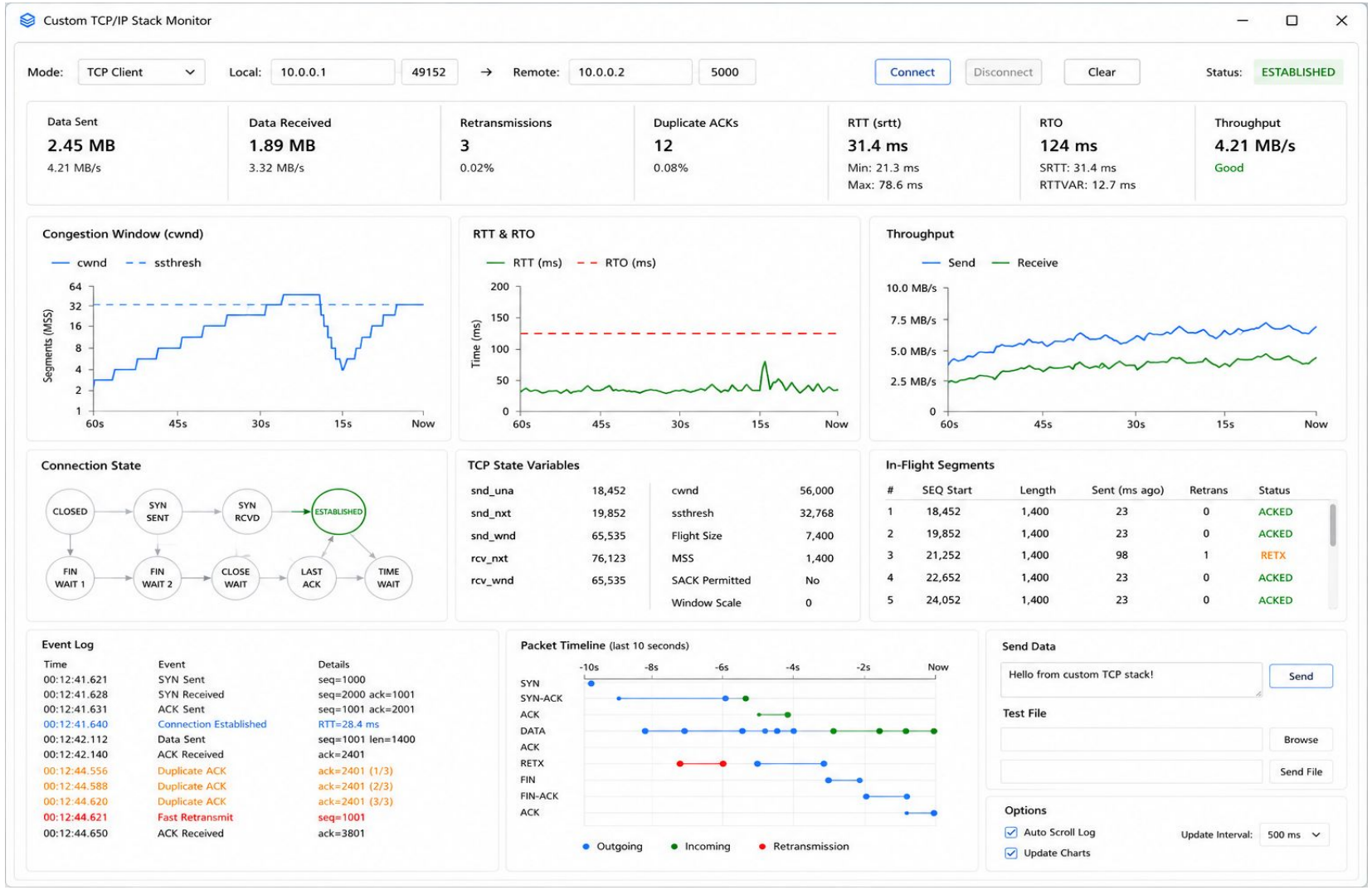
Connection lifecycle from open to close



The first implementation should prioritize the common path: LISTEN/active connect → ESTABLISHED → FIN handshake → TIME_WAIT/CLOSED. Rare RFC states can be added incrementally.

12. Observability UI

A simple desktop monitor is enough for the professor demo



Connection

Local/remote IP + ports, status, connect/disconnect.

TCP telemetry

cwnd, ssthresh, RTT, RTO, retransmissions, duplicate ACKs.

Experiment controls

Send data, clear logs, optional loss/latency controls.

Implementation rule

UI observes the C stack; it does not implement TCP itself.

13. How the UI connects to the C stack

Keep the networking core independent from presentation

1. C TCP stack

Maintains authoritative TCB, packet path, timers, retransmissions and congestion control.

2. Telemetry bridge

Exports read-only metrics/events at a small fixed interval or on state changes.

3. Desktop UI

Charts and tables visualize the latest metrics and event stream.

Recommended first implementation

```
C stack → localhost telemetry endpoint
or newline-delimited JSON / named pipe

Example: {"cwnd":5600,"rtt_ms":31.4,"retrans":3}
```

Why this separation is useful

- Core stack remains testable without the UI.
- UI can be rebuilt without touching packet logic.
- Professor can see protocol behavior in real time.
- An .exe can package the C stack and a lightweight desktop UI.

14. Experimental validation

Turn the project into a measurable networking experiment

Baseline 0% loss, normal latency	**Expected observation** Handshake, transfer and close complete.
Loss 1–5% packet loss	**Expected observation** Observe retransmissions, duplicate ACKs and cwnd reduction.
Delay Artificial RTT increase	**Expected observation** RTT/RTO estimation should adapt.
Burst loss Short high-loss interval	**Expected observation** Observe recovery and throughput drop.
Large transfer Multi-MB stream	**Expected observation** Measure throughput, in-flight data and stability.

Metrics to record

- Throughput (MB/s)
- cwnd and ssthresh over time
- RTT, RTTVAR and RTO
- Retransmissions
- Duplicate ACK count
- Bytes in flight
- Connection state transitions

Evidence for evaluation

Use the UI graph + packet capture + terminal logs for the same experiment run.

15. Implementation status: what must be fixed

Be technically honest in the professor review

Receive advertised window	**Required change** snd_wnd must be updated from incoming TCP headers; otherwise tcp_send can stall at zero usable window.
Blocking connect/ARP	**Required change** Packet reception should run in the main event loop rather than sleeping while waiting for network events.
Actual retransmission	**Required change** Inflight metadata must retain segment bytes/flags so timeout and fast retransmit can resend the exact segment.
Reno fast recovery	**Required change** Triple-duplicate-ACK handling should differ from timeout handling; do not reset cwnd to 1 MSS for fast recovery.
TCP header validation	**Required change** Validate data offset and lengths before using payload pointers.
Build completeness	**Required change** Include math.h for fabs() and link libm where required.

These are engineering corrections, not reasons to abandon the architecture. Fixing them makes the demonstration defensible.

16. 2–4 week execution plan

Build in dependency order

WEEK 1

Packet foundation

TAP, Ethernet, ARP, IPv4, ICMP, UDP
Verify with ping + UDP echo.

WEEK 2

TCP reliability

Event loop, handshake, ACKs, data transfer, FIN/close, receive/send windows.

WEEK 3

Reliability + Reno

Inflight bytes, retransmission, RTT/RTO, timeout response, duplicate ACKs, fast retransmit/recovery.

WEEK 4

Measurement + demo

Desktop monitor, loss/latency experiments, cwnd/RTT/throughput graphs, packet captures, documentation.

Critical dependency: do not add congestion-control complexity before handshake + reliable data transfer are genuinely working.

17. Professor demonstration sequence

A concise, evidence-driven demo

01	**Architecture**	Show the stack and explain why TAP gives control of L2 frames.
02	**Packet path**	Send one message and trace Application → TCP → IPv4 → ARP → Ethernet → TAP.
03	**Handshake**	Show SYN → SYN+ACK → ACK in the UI and packet capture.
04	**Data**	Transfer a measurable payload; show sequence/ACK movement.
05	**Loss**	Introduce controlled loss and show retransmission + cwnd response.
06	**Analysis**	Explain why cwnd changed, how RTO changed and what the graphs prove.

Demo rule

Show evidence at every claim: packet capture for packets, UI for metrics, code for implementation choices.

18. Likely professor questions

Prepare answers around mechanisms, not slogans

Why not use normal sockets?	Because normal sockets would hide TCP/IP processing inside the kernel; TAP exposes Ethernet frames so the project can implement the stack itself.
What happens when a packet is lost?	The receiver generates duplicate ACKs or the sender's RTO expires; the sender retransmits and reduces its congestion window according to the loss mechanism.
What is the difference between flow and congestion control?	Flow control protects the receiver via snd_wnd/rcv_wnd; congestion control protects the network via cwnd.
How do you know the packet belongs to a connection?	Match the TCP 4-tuple: local IP/port + remote IP/port.
How do you estimate RTT?	Measure ACK time for a non-retransmitted segment and update SRTT/RTTVAR; Karn's algorithm avoids ambiguous samples.
What makes your project measurable?	The stack exports cwnd, RTT/RTO, ACK/loss events, retransmissions and throughput, which can be correlated with packet captures.

19. Final system

What the finished project should demonstrate

Custom packet path

Application → TCP → IPv4 → ARP/Ethernet → TAP

Reliable transport

Sequence numbers, ACKs, buffers, retransmission and connection teardown.

Congestion control

cwnd, ssthresh, slow start, congestion avoidance and loss recovery.

Observability

Live UI, packet capture and experiments connect implementation to behavior.

The strongest final demonstration is not the UI itself. It is the chain of evidence:

C implementation — **Real TAP frames** — **TCP packet capture** — **Controlled loss** — **cwnd/RTT response** — **Measured throughput**

This turns the project from “a TCP implementation” into a reproducible networking systems experiment that can explain exactly what happens when packets move, arrive late, are lost, acknowledged and retransmitted.

Appendix A — Core source concepts

Directly aligned with the supplied implementation

```
typedef struct tcb {
    tcp_state_t state;
    uint32_t snd_una, snd_nxt, snd_wnd;
    uint32_t rcv_nxt, rcv_wnd;
    cc_state_t cc;
    rtt_estimator_t rtt;
} tcb_t;
```

```
typedef struct {
    cc_phase_t phase;
    uint32_t cwnd;
    uint32_t ssthresh;
    uint32_t dup_ack_count;
} cc_state_t;
```

Important implementation concepts

- 4-tuple connection lookup
- Sequence and acknowledgement numbers
- Inflight segment tracking
- RTO timer and exponential backoff
- Karn's algorithm for RTT sampling

Current source limitations to disclose

- TCP options are not implemented
- IPv4 fragmentation/reassembly is not implemented
- Current retransmission metadata does not yet retain bytes
- Blocking waits need to move to the event loop